lumen_verify.sh — guide

Revision: 1 **Last modified:** 2026-09-26T13:37:00Z

Source: `constitution/scripts/lumen/lumen_verify.sh` (109 lines: a bash wrapper, lines 1-50, and an embedded Python program, lines 50-108). Tests: `constitution/scripts/lumen/tests/test_lumen_verify.sh` (cases t1-t14, hermetic: a stub `lumen` binary, no network, no real index). Citations below are `lumen_verify.sh:<line>` unless another file is named.

Overview

Golden-query verification of a real Lumen semantic-search index (header, lines 2-8). It runs every golden natural-language query through the real `lumen search` CLI and checks that each expected ("gold") file appears among the top-k DISTINCT files. Queries whose gold file has an extension Lumen cannot index (type `unsupported`) must NOT be found: they document the capability gap instead of hiding it (lines 6-8, 74-75).

The verdict combines a recall threshold with a monotone baseline of KNOWN misses (lines 17-23, 86-91, 108):

- recall = PASS / (PASS + FAIL) over the in-scope, indexable (non-`unsupported`) queries (line 86-87);
- the run passes only if recall >= `--min-recall` AND every FAIL is listed in the baseline (a FAIL not listed is a NEW_MISS);
- a known miss that now passes is REMOVED from the baseline file (the tool only ever shrinks it); adding a miss is a deliberate human edit;
- an empty or erroring search is "could not look", never "absent" (lines 25-27, 68-71).

Prerequisites

- `bash`, `python3`.
- A Lumen binary: `--lumen <bin>`, otherwise the newest (`sort -V | tail -1`) of `$HOME/.claude*/plugins/cache/claude-plugins-official/lumen/*/bin/lumen-linux-amd64` (line 34). It must be executable, else exit 2 (line 46).
- A golden JSON file and an existing project directory (line 46).
- A working embedder and index for the project. The tool does not create or provision them: `lumen search` uses the caller's environment (`XDG_DATA_HOME`, `OLLAMA_HOST`, `LUMEN_*`) unchanged (lines 15-16).

Usage

```
lumen_verify.sh --golden <file.json> --project <root> [--k 5] [--n 20]
                [--lumen <bin>] [--out <dir>] [--scope-key in_tierA]
                [--min-recall 0.85] [--baseline <known_misses.json>] [--no-update-baseline]
```

Argument	Default	Meaning	Source
`--golden FILE`	required	Golden queries (format below)	lines 12-14, 37
`--project DIR`	required	Project root passed to `lumen search -p` and used as the working directory of each search	lines 37, 61-62
`--k N`	5	A query passes if a gold file is at rank <= N among distinct files	lines 33, 38, 77
`--n N`	20	Number of results requested from `lumen search -n`	lines 33, 38, 61
`--lumen BIN`	newest plu-gin-cache copy	Lumen binary	lines 34, 38
`--out DIR`	`mktemp -d`	Directory for `results.tsv` and `summary.txt`; created if missing	lines 39, 47-48
`--scope-key KEY`	`in_tierA`	A golden entry with `KEY: false` is out of scope and SKIPPED (unless its type is `unsupported`)	lines 39, 59-60
`--min-re-call X`	0.85	Recall threshold	lines 33, 40, 108
`--baseline FILE`	none	Known-misses file, `{"known_misses": {id: {reason, observed_rank}}}`; must exist	lines 40, 49, 54
`--no-update-baseline`	(updates)	Report TIGHTEN but do not rewrite the baseline	lines 41, 99-100
`-h`, `--help`		Print lines 2-32 of the script and exit 0	line 42
anything else		`unknown arg: <arg>`, exit 2	line 43

Golden file: a JSON list of `{id, type, q, gold, [<scope-key>]}` where `type` is `conceptual`, `structural` or `unsupported`, `gold` is a list of relative paths and a trailing `/` means "any file under that directory" (header lines 12-14, line 72). Queries are run in sorted `id` order (line 57).

Examples

Run with a STUB `lumen` (the one from the test: it emits `<result:file filename="...">` lines keyed on a query word) against an empty scratch project directory, never against a real Lumen index. The golden file held four queries: `a` (conceptual, gold at rank 2), `b` (conceptual, gold at rank 6), `u` (unsupported, gold absent), `s` (conceptual, `in_tierA: false`).

#	Command	Status
1	`lumen_verify.sh --help`	VERIFIED (ran; saw 31 lines, the last being `set -u`, exit 0)
2	`lumen_verify.sh` (no arguments)	VERIFIED (ran; saw `usage: need --golden FILE --project DIR and an executable lumen (got '<default binary path>')`, exit 2)
3	`lumen_verify.sh --bogus`	VERIFIED (ran; saw `unknown arg: --bogus`, exit 2)
4	Golden above, no baseline, `--k 5`	VERIFIED (ran; summary `k=5 PASS=2 FAIL=1 SKIP=1 ERROR=0 recall=0.500 min_recall=0.85 known_misses=0 new_misses=1 tighten=0`, line `NEW_MISS b FAIL 6 target.py`, exit 1; `results.tsv` rows `a PASS 2`, `b FAIL 6`, `s SKIP - ... out-of-scope`, `u PASS -`)
5	Same with a baseline listing `b`, `--min-recall 0.5`	VERIFIED (ran; `KNOWN_MISS b FAIL 6 target.py rank 6 at k=5`, `new_misses=0`, exit 0: a FAIL is tolerated when known and recall meets the threshold)
6	`--baseline /nonexistent`	VERIFIED (ran; `baseline not found: /nonexistent`, exit 2)
7	Same golden with `--k 6` and the baseline from example 5	VERIFIED (ran; `PASS=3 FAIL=0 ... recall=1.000 ... tighten=1`, line `TIGHTEN b known miss now passes -> removed from baseline`; the baseline file was rewritten)
8	Without `--out`	VERIFIED (ran; exit 1 with the same verdicts; an output directory `tmp.XXXXXXXXXX` was created under `$TMPDIR` and left there, see Edge cases; removed by hand)
9	`TMPDIR=<scratch> bash tests/test_lumen_verify.sh`	VERIFIED (ran; `PASS t1_top_k_hit` ... `PASS t14_default_threshold_0_85`, all 14 PASS, exit 0; the test makes its own `mktemp -d` and removes it; nothing left in the scratch `TMPDIR`)

#	Command	Status
10	Against a real Lumen index with a real embedder	NOT RUN (the brief forbids pointing lumen at the live index; a real run also re-indexes the project, see Side effects)

Outputs

- `<out>/results.tsv` : one row per golden query, tab separated: `id` , verdict, rank (or `-`), gold list joined by `,` , and the top-k files joined by spaces (or `out-of-scope` / `rc=... results=... <stderr>` for SKIP / ERROR rows). Sorted by id, no timestamps (lines 24-25, 60, 70, 79-82).
- `<out>/summary.txt` : one line, `k=<k> PASS=<n> FAIL=<n> SKIP=<n> ERROR=<n> recall=<r|n/a> min_recall=<x> known_misses=<n> new_misses=<n> tighten=<n>` , also printed on stdout (lines 83-93).
- stdout after the summary: one line per FAIL or ERROR row, `NEW_MISS` , `KNOWN_MISS` or `ERROR` followed by `id, verdict, rank, gold` and the baseline reason; then one `TIGHTEN` line per known miss that now passes (lines 94-99).
- The baseline file is rewritten (sorted keys, indent 1) when a known miss now passes and `--no-update-baseline` is absent (lines 100-105).

Verdicts: `PASS` ; `FAIL` (gold not in the top k, or, for `unsupported` , gold found); `SKIP` (out of scope); `ERROR` (search exited non-zero or returned no files) (lines 59-79).

Exit codes

Code	Meaning	Source
0	No ERROR, and (recall is `n/a` or recall >= `--min-recall`) and no NEW_MISS	line 108
1	Recall below the threshold, or at least one NEW_MISS	line 108
2	Usage error (missing golden/project/binary, unknown argument, baseline not found), OR any query ended in ERROR	lines 43, 46, 49, 106-107

The header (lines 25-26) says "Exit 0 = all PASS, 1 = at least one FAIL"; the code is the table above: a known, baselined FAIL does not make the run fail (example 5, and test t10). See Findings.

Edge cases

- **Empty result is an error.** A search that exits 0 but returns no `<result:file ...>` lines is an ERROR row and the run exits 2, not a FAIL (lines 63-71; test t3). A search that exits non-zero is the same (test t4).
- **Distinct files only.** Files are de-duplicated in result order before ranking, so several chunks of one file count once (lines 64-67).
- **Directory gold.** A gold entry ending in `/` matches any file under it (line 72).
- **`unsupported` queries** invert the verdict: PASS when the gold is absent, FAIL when it is found, and they are excluded from the recall computation, and can never be baselined as known misses (lines 74-75, 86, 89; tests t5, t6).
- **Out-of-scope entries** with `<scope-key>: false` are SKIPPED and not searched; an `unsupported` entry is never skipped (line 59).
- **`--k` larger than `--n`** cannot rank beyond `--n` results (`-n`, line 61); NOT RUN.
- **`ERROR` rows are counted separately** and any ERROR forces exit 2 even if recall is fine (lines 106-107).
- **Recall is `n/a`** when there is no in-scope indexable query; the threshold is then skipped (line 87, 108).

- **Each search has a 600 s timeout** (`timeout=600` , line 62); a timeout raises an uncaught Python exception (exit 1 from Python, no results file written). UNCONFIRMED: not run.
- **Default `--out` is not cleaned up.** Without `--out` , `mktemp -d` creates a directory under `$TMPDIR` that stays behind (line 47; example 8).
- **Golden entries with a missing key** (`id` , `type` , `q` , `gold`) raise an uncaught Python `KeyError` (lines 57-58, 88). UNCONFIRMED: not run.
- **Baseline rewrite is not atomic** and happens after the summary is written; the file is rewritten by `json.dump` with sorted keys, which may reorder a hand-formatted baseline (lines 104-105).

Side effects

`lumen search` runs EnsureFresh: it incrementally re-indexes changed files of `<project>` into the index selected by `XDG_DATA_HOME` (header lines 28-30). To keep a shared index untouched, point `XDG_DATA_HOME` at a copy. The script itself writes only the output directory and, on TIGHTEN, the baseline file.

Internal behaviour

1. Parse arguments and validate inputs; `exec` an embedded Python program (lines 33-50).
2. Load the baseline's `known_misses` (line 54); iterate golden queries sorted by id (line 57).
3. For each in-scope query run `lumen search -n <n> --summary --min-score -1 -p <project> <q>` with the project as working directory, extract `<result:file filename="...">` names (lines 55, 61-67).
4. Decide PASS/FAIL/ERROR (lines 68-79); write `results.tsv` and `summary.txt` (lines 80-85, 92).
5. Compute recall, NEW_MISS and TIGHTEN sets, print the report, optionally rewrite the baseline, then choose the exit code (lines 86-108).

Findings

1. **Header exit-code statement is inaccurate** (lines 25-26 vs 108): a FAIL that is a known miss does not produce exit 1.
2. `--help` **prints one code line**: `sed -n '2,32p'` (line 42) prints the header (lines 2-31) plus line 32, `set -u`.
3. **Header side-effects note is correct but easy to miss:** a verification run mutates the index selected by `XDG_DATA_HOME`.

Related

File	Relation
gen_lumenignore	Generates the `.lumenignore` that scopes what this verification can find
`tests/test_lumen_verify.sh`	The hermetic tests (stub binary)
owed_report	Reports whether this tool has tests and a guide

Last verified

2026-09-26 (source lines read on that date; examples 1-9 run with a stub `lumen` and the test in a scratch `TMPDIR`; no real index touched).